

Heterophily-Based Dynamic Hypergraph for Semisupervised Classification and Its Application in UAV Fault Diagnosis

Shaojun Liang , Ying Zheng , Senior Member, IEEE, and Housheng Su 

Abstract—Hypergraph neural network (HGNN) excels at representing intricate relationships within non-Euclidean infinite-dimensional spaces. However, traditional hypergraph frameworks are grounded in homophily assumption and encounter challenges in handling heterophily problem in semisupervised classification tasks. To tackle this problem, this article first analyzes the underlying reasons that limit homophily-based hypergraphs, and proposes a heterophily-based dynamic structure. Under this structure, a heterophily-based dynamic HGNN (HDHGNN) is then proposed for semisupervised classification. HDHGNN decomposes the hypergraph's triple pattern iterative process into two causally related subprocedures, within which attention mechanisms are incorporated to enable dynamic interactions. To evaluate the effectiveness of HDHGNN, a relative entropy-based heterophily degree evaluation method is first introduced, followed by comprehensive fault classification experiments on two real-world flight datasets from rotary-wing and fixed-wing unmanned aerial vehicles, with comparisons against 11 state-of-the-art methods.

Impact Statement—In real-world applications, relationships among diverse entities often exhibit higher-order interactions and pronounced heterophilic characteristics. While hypergraphs generalize simple graphs beyond pairwise relations, conventional HGNNs still rely on the homophily assumption, limiting their effectiveness in heterophilic scenarios. The development of HGNNs tailored to heterophily has thus emerged as a critical research frontier with significant practical implications. The HDHGNN proposed in this article overcomes this limitation, achieving improvements of up to 2.29% on highly heterophilic datasets and

up to 5.75% on homophilic benchmarks. These results highlight the robustness and adaptability of HDHGNN across varying degrees of heterophily, underscoring its potential for real-world applications in complex and heterophilic scenarios.

Index Terms—Graph neural network (GNN), heterophily problem, hypergraph, semisupervised classification.

I. INTRODUCTION

GRAPH neural networks (GNNs), as a class of deep learning models, have attracted significant attention recently. By representing data as nodes and their relationships as edges on a graph, GNNs are capable of expressing intricate relationships in non-Euclidean infinite-dimensional spaces [1]. Consequently, GNNs have been widely applied in diverse domains, such as recommendation systems [2], knowledge graphs [3], and semisupervised classification tasks [4].

GNNs can be categorized into low-order and high-order graphs based on the complexity of the relationships to be modeled. The low-order graphs, often referred to as simple graphs, connect pairs of nodes with a single edge and follow a “node-node” feature propagation pattern [1]. Specifically, this pairwise pattern first aggregates messages from neighboring nodes and then updates the target node's features accordingly [5]. In practical scenarios, node relationships often exhibit complex patterns that cannot be fully captured by pairwise patterns. To address this limitation, a form of high-order GNNs, known as hypergraph neural networks (HGNNs) [6] has been introduced. In the HGNNs, multiple nodes can be interconnected through a single hyperedge, enabling relationships of arbitrary cardinality. The feature propagation process follows a “node-hyperedge-node” triple pattern, where messages from neighboring nodes are first aggregated onto hyperedges, then these hyperedge-level representations are propagated back to the nodes [7]. This abstract architecture endows HGNNs with enhanced expressive power for modeling complex relationships, and has garnered significant research interest. Representative frameworks such as DHG [7] and PyG [8], along with methods including HGNN [6], HGNNP [7], HyperGCN [9], UniGCN [10], and UniGIN [10] have been developed and widely applied across diverse domains.

Most hypergraph methods operate under the homophily assumption—that is, connected nodes tend to share similar features or labels [11]. Under this assumption, semisupervised

Received 16 December 2025; revised 26 January 2026; accepted 26 February 2026. Date of publication 12 March 2026; date of current version 31 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62425602, Grant U25B2078, Grant 62561160100, and Grant 62273159; in part by Hubei Province International Science and Technology Cooperation Project under Grant 2024EHA033; in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2025A1515010134; and in part by the Natural Science Foundation of Hubei Province of China under Grant 2025AFA027. This article was recommended for publication by Associate Editor Feng Xia upon evaluation of the reviewers' comments. (Corresponding author: Ying Zheng.)

Shaojun Liang is with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan 430074, China, and also with Army Engineering University, Wuhan 430075, China.

Ying Zheng is with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan 430074, China, and also with Shenzhen Huazhong University of Science and Technology Research Institute, Shenzhen 518057, China (e-mail: zyhid@mail.hust.edu.cn).

Housheng Su is with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan 430074, China.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TAI.2026.3672585>, provided by the authors.

Digital Object Identifier 10.1109/TAI.2026.3672585

classification typically proceeds by aggregating features from neighboring nodes to update a target node's representation. However, this paradigm breaks down in heterophilic scenarios, where adjacent nodes often exhibit dissimilar label distributions [12]. Such scenarios are widely observed in real-world systems: for instance, in biomedical applications, functionally distinct amino acids interact to form complex protein structures; in UAV fault diagnosis, samples from different classes exhibit significant overlap in feature space, demonstrating a strong heterophilic distribution. In these scenarios, conventional homophily-based aggregation paradigm indiscriminately mixes features from neighbors of disparate classes, thereby corrupting node representations and causing severe performance degradation in semisupervised classification. Recent studies have further shown that graph-based methods often underperform even simple nongraph baselines such as multilayer perceptrons [11], and that heterophily is more pronounced in hypergraphs than in simple graphs [13].

Consequently, the heterophily problem remains a critical challenge in graph-based classification tasks, particularly in hypergraphs, which has been recognized as a critical research frontier with important practical relevance [5], [14]. Currently, numerous approaches have been proposed in the domain of simple graphs to address this problem, broadly falling into three categories: 1) label-aware long-range aggregation (e.g., GOAL [15], OrderedGNN [16], and EvenNet [17]); 2) architecture refinement for enhanced expressiveness (e.g., RFA-GNN [18], ES-GNN [19], and CAGNNs [20]); and 3) hybrid approaches that combine both strategies (e.g., H2GCN [11], DeformableGCN [21]).

However, the “node-hyperedge-node” triple propagation pattern inherent to hypergraphs is fundamentally distinct from the pairwise “node-node” interaction paradigm in simple graphs. Consequently, existing heterophily-aware methods designed for simple graphs are not directly applicable to hypergraph architectures. To the best of our knowledge, hypergraph-based models explicitly tailored to address heterophily remain in their early stages of development. Current hypergraph approaches, such as UniGAT [10], UniSAGE [10], DHGNN [22], and HNH [23], primarily rely on dynamic weight assignment during aggregation to modulate message passing. Others, including SCHG [24], DualHGNN [25], MD-HGNN [26], and HyperGT [27], aim to improve semisupervised classification by capturing richer structural and contextual information through advanced hypergraph modeling. Additional efforts leverage energy minimization (e.g., I²HGNN [28]), low-rank representation (e.g., NSHLRR [29]), or structural refinement (e.g., HCHA [30], HyperMSG [31]) to enhance performance. Critically, these methods are fundamentally rooted in the homophily assumption; thus, any performance gains observed under heterophilic conditions are largely incidental rather than attributable to deliberate heterophily-aware design.

Only a limited number of hypergraph methods have been developed to address the challenges posed by heterophily in semisupervised classification tasks. For instance, Wang et al. [32] introduced ED-HNN, which integrates star expansion techniques with message-passing protocols to explicitly capture

heterogeneous relational patterns in hypergraphs. Zou et al. [33] proposed UniG-Encoder, a unified framework capable of simultaneously modeling homophilic and heterophilic interactions within hybrid graph-hypergraph structures. Nguyen et al. [34] further advanced this domain by formulating SHNs, a theoretical framework grounded in cellular sheaf theory that concurrently addresses over-smoothing and homophily-induced limitations through topological abstraction.

To address these challenges, we conduct a careful analysis of the homophily-based static message-passing mechanism in conventional HGNNs, and identify the fundamental cause of their poor performance in heterophilic scenarios. Building on this insight, we first propose a heterophily-based dynamic architecture that replaces the homophily assumption with a heterophily-oriented design at the structural level. Then, we introduce Heterophily-based Dynamic HGNN (HDHGNN) method, which decouples the traditional triple propagation pattern into two adaptive, dynamically coordinated procedures. The key contributions can be summarized as follows:

- 1) A heterophily-based dynamic framework is proposed to address the heterophily problem in hypergraphs. Departing from the homophily-based static paradigm, the framework introduces two opposing forces, mutual attraction and mutual repulsion, and dynamically adjusts their strengths to enable adaptive interactions between nodes and hyperedges, thereby enhancing its capacity to model heterophilic distributions at the framework level.
- 2) A heterophily-aware hypergraph method, HDHGNN, is derived from the proposed dynamic framework. HDHGNN decomposes propagation into forward and reverse steps and introduces force-aware attention mechanisms to accommodate varying degrees of heterophily, thereby broadening the applicability of hypergraph learning and enhancing the model's robustness in real-world settings with complex heterophilic scenarios.
- 3) A relative entropy-based heterophily degree (RD) metric is proposed accompanied by a detailed mathematical analysis, providing a more rational and fine-grained quantification of heterophily in hypergraphs.

The subsequent sections are organized as follows. Section II introduces the theoretical groundwork, while Section III presents the main innovation of the proposed framework and HDHGNN method. Section IV is devoted to experimental evaluation, and the conclusion is given in Section V.

II. PRELIMINARIES

A. Problem Formulation

Let $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathcal{S}\}$ represent a hypergraph, where $\mathcal{V}$ denotes the set of N nodes, $\mathcal{E}$ denotes the set of E hyperedges, and $\mathcal{S}$ denotes the structure of the hypergraph, i.e., the manner by which nodes are interconnected. Let $\mathbf{X} = [x_1, x_2, \dots, x_N]^T \in \mathbb{R}^{N \times d}$ denote the feature matrix, where each row corresponds to a d -dimensional feature vector of a node. And let $\mathcal{Y} = \{1, 2, \dots, C\}$ represent the set of C category labels. Suppose each node $v_i \in \mathcal{V}$ is associated with a unique label $y_i \in \mathcal{Y}$. Thereby, the objective of semisupervised node classification on a hypergraph

is to learn a mapping $\mathcal{M}: \mathcal{V} \rightarrow \mathcal{Y}$ given a labeled subset $\mathcal{T}_\mathcal{V} = \{(v_1, y_1), (v_2, y_2), \dots, (v_N, y_N)\}$ and the hypergraph structure $\mathcal{S}$, aiming to predict the labels of the remaining unlabeled nodes.

B. Hypergraph Theories

Specifically, the hypergraph structure $\mathcal{S}$ is characterized by the hyperedge weight matrix $\mathbf{W}$ and the incidence matrix $\mathbf{H}$. The matrix $\mathbf{W} = \text{diag}(w_1, w_2, \dots, w_E) \in \mathbb{R}^{E \times E}$ is a diagonal matrix where each element w_i denotes the weight of hyperedge $e_i \in \mathcal{E}$. The incidence matrix $\mathbf{H} \in \mathbb{R}^{N \times E}$ describes the relationship between nodes and hyperedges, with its (i, j) th element $h_{i,j} \in [0, 1]$ indicating the strength of the association between node v_i and hyperedge e_j . The formulation of $h_{i,j}$ can be expressed as [35]

$$h_{i,j} = \begin{cases} 1, & \text{if } v_i \in e_j \\ 0, & \text{else} \end{cases} \quad (1)$$

where $v_i \in e_j$ denotes node v_i is on hyperedge e_j .

To construct the incidence matrix $\mathbf{H}$, the KNN method is commonly employed to determine node relationships within hyperedges. Specifically, each node is treated as a central node, and its k nearest neighbors are selected to form a corresponding hyperedge. Consequently, the number of hyperedges equals the number of nodes, i.e., $E = N$. Different choices of k lead to varying levels of connectivity in the hypergraph, thereby affecting the level of heterophily.

Conventional hypergraph methods rely on the homophily assumption that nodes on the same hyperedge tend to share the same label [18]. Under this assumption, a typical feature propagation process is formulated as follows [7]

$$\begin{aligned} \mathbf{X}^{(l+1)} &= \sigma \left(\mathbf{D}_v^{-1} \mathbf{H} \mathbf{W} \mathbf{D}_e^{-1} \mathbf{H}^\top \mathbf{X}^{(l)} \Theta^{(l)} \right) \\ &= \sigma \left(\mathcal{S}_d \mathbf{X}^{(l)} \Theta^{(l)} \right) \end{aligned} \quad (2)$$

where $\mathbf{X}^{(l)}$ is the feature matrix in the l th iteration, initialized as $\mathbf{X}^{(0)} = \mathbf{X}$; $\mathbf{D}_v = \text{diag}(D_{v_1}, D_{v_2}, \dots, D_{v_N}) \in \mathbb{R}^{N \times N}$ denotes the node degree matrix, with each diagonal element calculated by $D_{v_i} = \sum_{j=1}^E w_j h_{i,j}$; $\mathbf{D}_e = \text{diag}(D_{e_1}, D_{e_2}, \dots, D_{e_E}) \in \mathbb{R}^{E \times E}$ denotes the hyperedge degree matrix, with each diagonal element calculated by $D_{e_i} = \sum_{j=1}^N h_{j,i}$; $\mathbf{W}$ is an identity matrix in the conventional hypergraph setting; $\Theta^{(l)}$ is the learnable parameter matrix in the l th iteration; σ denotes the nonlinear activation function.

It is important to note that $\mathcal{S}_d = \mathbf{D}_v^{-1} \mathbf{H} \mathbf{W} \mathbf{D}_e^{-1} \mathbf{H}^\top$ is an asymmetric matrix. Furthermore, $\mathcal{S}_d$ remains constant across layers and incorporates all structural components. Therefore, we refer to it as the static directed structure of hypergraph.

III. METHODOLOGY

A. Beyond Homophily: Heterophily-Based Dynamic Structure

To explore the difference between homophily-based and heterophily-based hypergraphs, we present a toy example with only three nodes, as shown in Fig. 1(a). This hypergraph contains two hyperedges, e_1 and e_2 , which are centered on nodes

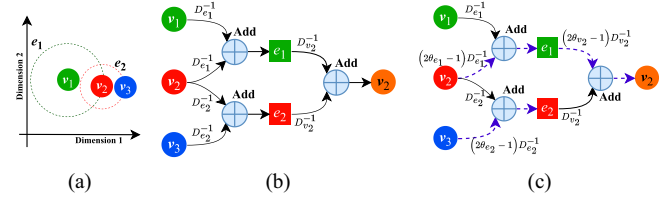


Fig. 1. Toy example illustrating the differences between homophily-based and heterophily-based hypergraph aggregation processes. (a) Example with three nodes. (b) Homophily-based hypergraph with static structure. (c) Heterophily-based hypergraph with dynamic structure.

v_1 and v_2 , respectively. For a homophily-based hypergraph governed by (2), the matrix $\Theta^{(l)}$ performs feature transformation at each node in the l th layer, without involving feature interactions among different nodes. Therefore, to focus exclusively on the structure-determined feature interactions, we set $\Theta^{(l)}$ to be an identity matrix and omit the nonlinear activation function. Consequently, the propagation mechanism can be formulated as: $\mathbf{X}^{(l+1)} = \mathcal{S}_d \mathbf{X}^{(l)} \mathbf{I}^{(l)}$. Accordingly, the node interaction in each layer can be viewed as a two-layer fully connected network with static weights and no activation function. To illustrate this point without loss of generality, we examine node v_2 as an example. Its feature interaction procedure with other nodes in the l th layer is illustrated in Fig. 1(b), which can be expressed as

$$\begin{aligned} x_2^{(l+1)} &= D_{v_2}^{-1} \left(\varepsilon_1^{(l)} + \varepsilon_2^{(l)} \right) \\ \text{s.t. } \begin{cases} \varepsilon_1^{(l)} = D_{e_1}^{-1} \left(x_1^{(l)} + x_2^{(l)} \right) \\ \varepsilon_2^{(l)} = D_{e_2}^{-1} \left(x_2^{(l)} + x_3^{(l)} \right) \end{cases} \end{aligned} \quad (3)$$

where $\varepsilon_1^{(l)}$ and $\varepsilon_2^{(l)}$ represent the features of hyperedges e_1 and e_2 in the l th layer, respectively.

The aforementioned process reveals that in conventional homophily-based hypergraphs with static structures, each node's updated feature vector is calculated as a fixed weighted sum of its neighbors' features. This aggregation mechanism can be interpreted as a "force of mutual attraction" among neighboring nodes, which inevitably drives their feature representations toward convergence. Such behavior fundamentally contradicts the objective of heterophilic classification task, where connected nodes often belong to different categories and must remain distinguishable.

In heterophilic settings, an ideal interaction procedure should pull a node's features closer to those of the same class while pushing them apart from nodes of other classes, even if they are topologically adjacent. We therefore argue that, alongside mutual attraction, a complementary "force of mutual repulsion" is essential to preserve inter-class separability. The dynamic interplay between these two opposing forces offers a more principled strategy for handling heterophily. Motivated by this insight, we design a heterophily-based dynamic structure, as illustrated in Fig. 1(c). The corresponding feature interaction

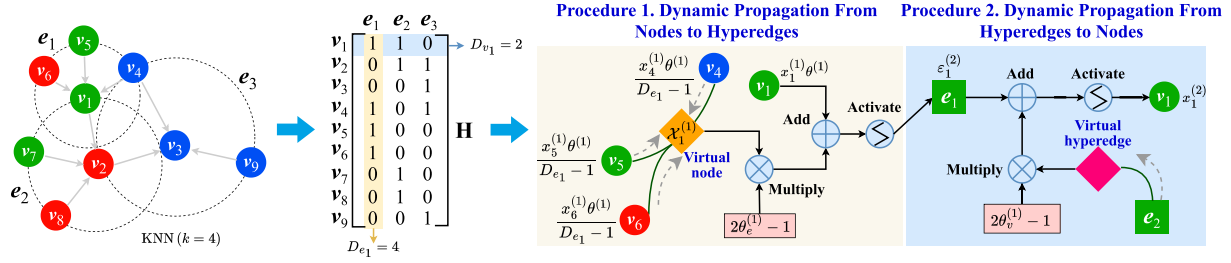


Fig. 2. Node-wise aggregation procedures of HDHGNN. The nine nodes belong to three classes (distinguished by color). three hyperedges are constructed via the KNN method ($k = 4$) centered at node v_1 , v_2 , and v_3 , respectively, yielding an incidence matrix $\mathbf{H} \in \mathbb{R}^{9 \times 3}$. Procedures 1 and 2 illustrate the detailed feature update process for node v_1 .

procedure for node v_2 can then be formulated as

$$\begin{aligned} x_2^{(l+1)} &= D_{v_2}^{-1} \varepsilon_2^{(l)} + (2\theta_{v_2} - 1) D_{v_2}^{-1} \varepsilon_1^{(l)} \\ \text{s.t. } \begin{cases} \varepsilon_1^{(l)} &= D_{e_1}^{-1} x_1^{(l)} + (2\theta_{e_1} - 1) D_{e_1}^{-1} x_2^{(l)} \\ \varepsilon_2^{(l)} &= D_{e_2}^{-1} x_2^{(l)} + (2\theta_{e_2} - 1) D_{e_2}^{-1} x_3^{(l)} \end{cases} \end{aligned} \quad (4)$$

where θ_{e_1} , θ_{e_2} and $\theta_{v_2} \in [0, 1]$ are adjustable parameters. When $\theta_{e_1} = \theta_{e_2} = \theta_{v_2} = 1$, this formula reduces to (3), adapting to the most extreme homophilic situation, under which it aims to make the feature distributions of neighboring nodes increasingly similar. In contrast, when $\theta_{e_1} = \theta_{e_2} = \theta_{v_2} = 0$, the procedure adapts to the most extreme heterophilic situation, emphasizing dissimilarity among neighbors and thereby encouraging divergence in their feature distributions. Intermediate values of these parameters allow the procedure to capture more complex distribution patterns.

Equation (4) introduces a flexible formulation capable of capturing both homophilic and heterophilic patterns. Nevertheless, it is limited to this simplified case where each hyperedge involves only two nodes. A more general extension is presented in the following section.

B. Heterophily-Based Dynamic Hypergraph Method

To instantiate a heterophily-aware hypergraph method that respects the dynamic structure introduced in the prior section, we decompose the feature update processes into two causally ordered dynamic subprocedures: forward “node-to-hyperedge” propagation, followed by reverse “hyperedge-to-node” propagation.

1) *Dynamic Propagation From Nodes to Hyperedges*: As depicted in Procedure 1 of Fig. 2, the “node-to-hyperedge” aggregation procedure aggregates features from neighboring nodes onto the shared hyperedges. A hyperedge e_i connects multiple nodes, including a central node v_i and its neighbors. Due to the distinct roles played by these nodes within the hyperedge, the features of neighboring nodes are first individually aggregated onto a virtual node, as

$$\mathcal{X}_i^{(l)} = (D_{e_i} - 1)^{-1} \sum_{v_j \in e_i \setminus v_i} x_j^{(l)} \theta^{(l)} \quad (5)$$

where $\theta^{(l)}$ denotes a learnable parameter used for feature dimension transformation, and $\mathcal{X}_i^{(l)}$ represents the aggregated feature of the virtual node in the l th iteration.

The virtual node serves as a local centroid that captures the collective characteristics of its neighbors prior to direct interaction with the central node itself. By abstracting neighboring nodes into a virtual node, this approach not only maintains a clear role distinction between the central node and its neighbors but also transforms the original one-to-many relationship into a simplified one-to-one interaction. This transformation establishes a structured basis for the subsequent introduction of parameters to modulate the pair-wise relationship, thereby enabling dynamic control over the balance between homophily and heterophily in the model.

Building upon this design, the dynamic aggregation from nodes to a hyperedge e_i in the l th iteration is formulated as

$$\varepsilon_i^{(l+1)} = \sigma \left(x_i^{(l)} \theta^{(l)} + (2\theta_{e_i}^{(l)} - 1) \mathcal{X}_i^{(l)} \right) \quad (6)$$

where $\varepsilon_i^{(l+1)}$ denotes the aggregated feature of hyperedge e_i after the l th iteration; $\theta_{e_i}^{(l)} \in [0, 1]$ is a learnable parameter employed to dynamically adjust the aggregation procedure. Specifically, when $\theta_{e_i}^{(l)} = 1$, (6) leans toward the homophily assumption; while when $\theta_{e_i}^{(l)} = 0$, it tends to support the heterophily assumption.

To enhance computational efficiency, the matrix form of (6) can be expressed as

$$\mathbf{E}^{(l+1)} = \sigma \left(\left(\mathbf{I} + 2\Theta_e^{(l)} \bar{\mathbf{D}}_e^{-1} \bar{\mathbf{H}}^\top - \bar{\mathbf{D}}_e^{-1} \bar{\mathbf{H}}^\top \right) \mathbf{X}^{(l)} \Theta^{(l)} \right) \quad (7)$$

where $\mathbf{E}^{(l+1)}$ denotes the hyperedge feature matrix after the l th iteration; $\bar{\mathbf{H}} = \mathbf{H} - \text{diag}(\mathbf{H})$ represents the incidence matrix with zero diagonal elements; $\bar{\mathbf{D}}_e$ is the hyperedge degree matrix associated with $\bar{\mathbf{H}}$; and $\Theta_e^{(l)} \in \mathbb{R}^{E \times E}$ is a diagonal matrix whose i th diagonal element is equal to $\theta_{e_i}^{(l)}$. Specifically, $\Theta_e^{(l)}$ can be calculated through the following attention mechanism:

$$\begin{aligned} \Theta_e^{(l)} &= \text{diag} \left\{ \sigma \left(\tilde{\mathbf{X}}^{(l)} \mathbf{W}_{e1}^{(l)} \right) \right\} \mathbf{W}_{e2}^{(l)} \\ \text{s.t. } \tilde{\mathbf{X}}^{(l)} &= \left[\mathbf{X}^{(l)} \Theta^{(l)}, \bar{\mathbf{D}}_e^{-1} \bar{\mathbf{H}}^\top \mathbf{X}^{(l)} \Theta^{(l)} \right] \end{aligned} \quad (8)$$

where $\mathbf{W}_{e1}^{(l)} \in \mathbb{R}^{2h \times h}$ and $\mathbf{W}_{e2}^{(l)} \in \mathbb{R}^{h \times 1}$ are learnable parameters with h denoting the dimensionality of hidden layers; $[\cdot]$ represents the horizontal concatenation operation of matrices; and $\text{diag} \{ \cdot \}$ denotes the operation of transforming a vector into a diagonal matrix.

Notably, the i th row vector of the concatenated matrix $\mathbf{X}^{(l)}$ in (8) can be expressed as $[x_i^{(l)}, \mathcal{X}_i^{(l)}]$. This vector integrates features from the central node v_i and its neighbors in the l th layer, capturing the heterophilic distribution around v_i . It thus serves as the basis for deriving the adjustable parameter $\Theta_e^{(l)}$.

2) *Dynamic Propagation From Hyperedges to Nodes*: The subsequent “hyperedge-to-node” subprocedure aggregates the hyperedge features back to their corresponding nodes, as illustrated in Procedure 2 of Fig. 2. Analogous to the “node-to-hyperedge” procedure in (5) and (6), this dynamic aggregation procedure at the l th iteration can be formulated as

$$x_i^{(l+1)} = \sigma \left(\varepsilon_i^{(l+1)} + \frac{2\theta_{v_i}^{(l)} - 1}{D_{v_i} - 1} \sum_{e_j \ni v_i \setminus e_i} \varepsilon_j^{(l+1)} \right) \quad (9)$$

where $\theta_{v_i}^{(l)} \in [0, 1]$ serves as a learnable aggregation parameter in the l th iteration. The term $1/(D_{v_i} - 1) \sum_{e_j \ni v_i \setminus e_i} \varepsilon_j^{(l+1)}$ acts as a virtual hyperedge feature aggregated across all hyperedges incident to v_i except e_i , playing a role analogous to that of the virtual node introduced in (5).

Similar to (6), when $\theta_{v_i}^{(l)} = 1$ or $\theta_{v_i}^{(l)} = 0$, the above formula adopts the pure homophilic and heterophilic assumptions, respectively. The key distinction between the procedures described in (6) and (9) lies in their aggregation direction: the former dynamically aggregates node features onto a shared hyperedge, whereas the latter dynamically aggregates hyperedge features onto a shared node. Nevertheless, due to the directed structure of the underlying graph, the two procedures are not symmetric in nature, hence do not constitute mutual inverses.

The matrix formulation of (9) can be derived as

$$\mathbf{X}^{(l+1)} = \sigma \left(\left(\mathbf{I} + 2\Theta_v^{(l)} \bar{\mathbf{D}}_v^{-1} \bar{\mathbf{H}} - \bar{\mathbf{D}}_v^{-1} \bar{\mathbf{H}} \right) \mathbf{E}^{(l+1)} \right) \quad (10)$$

where $\bar{\mathbf{D}}_v$ is the node degree matrix associated with $\bar{\mathbf{H}}$, and $\Theta_v^{(l)} \in \mathbb{R}^{N \times N}$ is a diagonal matrix given by

$$\begin{aligned} \Theta_v^{(l)} &= \text{diag} \left\{ \sigma \left(\left(\tilde{\mathbf{E}}^{(l)} \mathbf{W}_{v1}^{(l)} \right) \mathbf{W}_{v2}^{(l)} \right) \right\} \\ \text{s.t. } \tilde{\mathbf{E}}^{(l)} &= \left[\mathbf{E}^{(l+1)}, \bar{\mathbf{D}}_v^{-1} \bar{\mathbf{H}} \mathbf{E}^{(l+1)} \right] \end{aligned} \quad (11)$$

where $\mathbf{W}_{v1}^{(l)} \in \mathbb{R}^{2h \times h}$ and $\mathbf{W}_{v2}^{(l)} \in \mathbb{R}^{h \times 1}$ are learnable parameters.

3) *Overall Framework and Complexity Analysis*: By integrating the dynamic aggregation procedures of “node-to-hyperedge” and “hyperedge-to-node,” as formulated in (7) and (10), respectively, the “node-to-node” dynamic aggregation process of HDHGNN can be expressed as

$$\begin{aligned} \mathbf{X}^{(l+1)} &= \sigma \left(\tilde{\mathcal{S}}_{e2n}^{(l)} \sigma \left(\tilde{\mathcal{S}}_{n2e}^{(l)} \mathbf{X}^{(l)} \Theta^{(l)} \right) \right) \\ \text{s.t. } \begin{cases} \tilde{\mathcal{S}}_{e2n}^{(l)} = \mathbf{I} + \left(2\Theta_v^{(l)} - \mathbf{I} \right) \bar{\mathbf{D}}_v^{-1} \bar{\mathbf{H}} \\ \tilde{\mathcal{S}}_{n2e}^{(l)} = \mathbf{I} + \left(2\Theta_e^{(l)} - \mathbf{I} \right) \bar{\mathbf{D}}_e^{-1} \bar{\mathbf{H}}^\top \end{cases} \end{aligned} \quad (12)$$

where $\tilde{\mathcal{S}}_{e2n}^{(l)}$ and $\tilde{\mathcal{S}}_{n2e}^{(l)}$ can be regarded as the dynamically learned directed structures associated with the “hyperedge-to-node” and “node-to-hyperedge” subprocedures, respectively.

Typically, the output of the final layer in (12) is fed into a fully connected layer, followed by the cross-entropy loss calculation with respect to the ground-truth labels. To provide a clear illustration of the overall architecture, the framework of HDHGNN is depicted in Fig. 3.

a) *Complexity analysis*: The proposed method comprises two phases in each iteration. For the first phase, as illustrated in (5), the features of neighboring nodes on a shared hyperedge undergo dimensional transformation and are subsequently aggregated into a virtual node feature, with a computational cost of $\mathcal{O}((D_{e_i} - 1)dh)$. Further, as described in (6), the central node’s feature is dimensionally transformed and combined with the virtual node feature to generate the feature of a hyperedge, requiring $\mathcal{O}(dh + 2h)$. Consequently, the total complexity for aggregating features of a single hyperedge is $\mathcal{O}(D_{e_i}dh + 2h)$. For E hyperedges in the graph, the overall complexity for this phase is $\mathcal{O}(\sum_{i=1}^E D_{e_i}dh + 2h)$. In the second phase, as illustrated in (9), aggregating hyperedge features to a central node v_i involves $\mathcal{O}(D_{v_i}dh + 2h)$ operations. For N central nodes in the graph, the total complexity for this phase becomes $\mathcal{O}(\sum_{i=1}^N D_{v_i}dh + 2h)$.

Under the KNN-based hypergraph construction, we have $\sum_{i=1}^E D_{e_i} = \sum_{i=1}^N D_{v_i} = Nk$. Thus, the overall complexity per iteration is $\mathcal{O}(2Nkdh + 4h)$. Given that $2Nkd \gg 4$, the complexity reduces to $\mathcal{O}(2Nkdh)$. This linear dependency on N indicates that the HDHGNN’s complexity scales linearly with the node number, ensuring efficient scalability.

C. Relative Entropy-Based Heterophily Degree Metric

As noted in [36], conventional heterophily metrics, such as node homophily, edge homophily, and class homophily, are commonly employed to quantify the heterophily degree (HD) of a graph. Under these conventional HD metrics, each node categorizes its neighbors into a binary partition: either sharing the same class label as the node itself or belonging to a different class. This coarse dichotomy overlooks the rich diversity among distinct classes within the “different-class” group, thereby limiting the ability of such metrics to capture fine-grained heterophilic structures that commonly arise in real-world graphs. As shown in Fig. 4(a), conventional HD metrics classify neighbors of node v_1 into only two groups: those sharing v_1 ’s label (green) and all others (black), ignoring the influence of multi-class label distributions. To better characterize the heterophilic patterns, we introduce the RD metric as illustrated in Fig. 4(b). First, to provide a formal representation of the heterophilic distribution associated with node v_i , we define a vector $\mathcal{H}_i \in \mathcal{R}^C$ as

$$\mathcal{H}_i[c] = \sum_{j=1}^N s_{i,j} \mathbb{I}(y_j = c), \quad \forall c \in \{1, 2, \dots, C\} \quad (13)$$

where $\mathcal{H}_i[c] \in [0, N]$ denotes the c th element in vector $\mathcal{H}_i$.

For a dataset comprising C categories, maximal heterophily is achieved when the data is evenly distributed across all categories. In this scenario, the heterophilic distribution around each

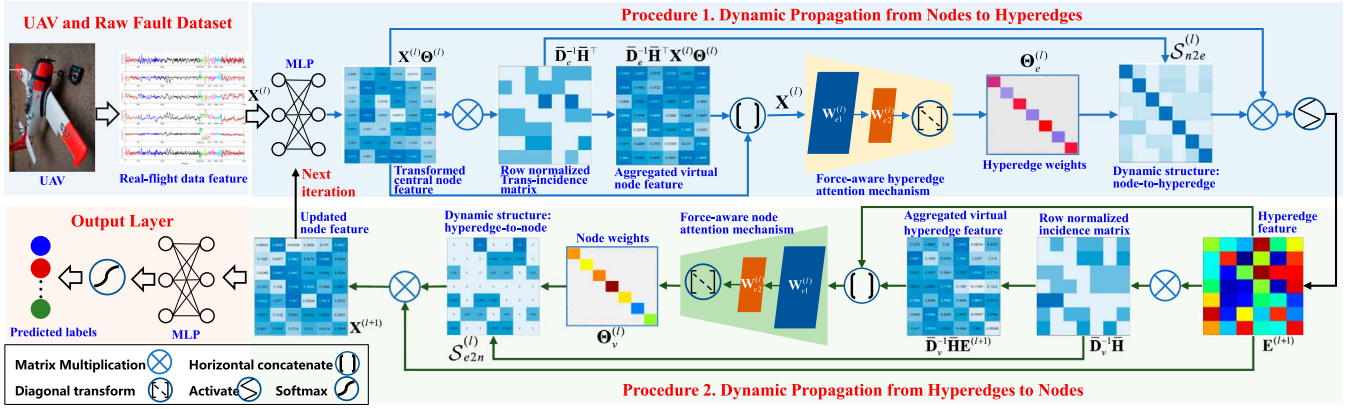


Fig. 3. Framework of HDHGNN. The model alternates between two dynamic propagation procedures: (1) node-to-hyperedge, where central node features and aggregated neighboring node (virtual node) features are concatenated and weighted via attention to form the dynamic incidence matrix $\tilde{S}_{n2e}^{(l)}$ for hyperedge feature generation; and (2) hyperedge-to-node, which symmetrically propagates hyperedge features back to nodes using $\tilde{S}_{e2n}^{(l)}$. After stacking multiple layers, the model performs final node embeddings, which are fed into an output layer for classification.

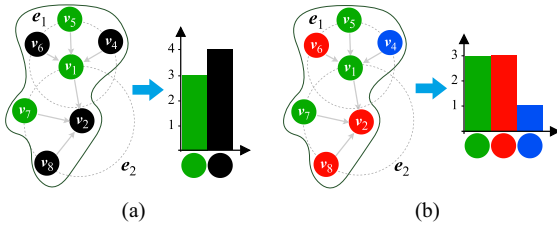


Fig. 4. Illustration of heterophily around node v_1 under two metrics. Dashed curves denote hyperedges e_1 and e_2 , both containing v_1 ; thus all shown nodes are one-hop neighbors of v_1 . (a) Conventional HD metric treats heterophily as binary-classes (green: v_5, v_7 versus different-class (black: v_2, v_4, v_6, v_8) ignoring diversity within the latter. (b) Proposed RD metric captures intraheterophilic variation by distinguishing label subgroups (e.g., blue v_4 versus red v_2, v_6, v_8), yielding a more nuanced heterophily profile.

node, denoted as $\mathcal{H}_\alpha$, is equal, which is expressed as

$$\mathcal{H}_\alpha[c] = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i = c), \quad \forall c \in \{1, 2, \dots, C\} \quad (14)$$

where $\mathcal{H}_\alpha[c] \in (0, 1]$ denotes the c th element in the vector $\mathcal{H}_\alpha$, representing the proportion of nodes belonging to category c relative to the total number of nodes N .

Utilizing the aforementioned distributions of $\mathcal{H}_i[c]$ and $\mathcal{H}_\alpha[c]$, the relative entropy of the heterophilic distribution for node v_i can be calculated via

$$\mathcal{K}(\hat{\mathcal{H}}_i || \mathcal{H}_\alpha) = \begin{cases} \sum_{c=1}^C \hat{\mathcal{H}}_i[c] \log \left(\frac{\hat{\mathcal{H}}_i[c]}{\mathcal{H}_\alpha[c]} \right), & \text{if } \hat{\mathcal{H}}_i[c] \in (0, 1] \\ 0, & \text{else } \hat{\mathcal{H}}_i[c] = 0 \end{cases} \quad (15)$$

where $\hat{\mathcal{H}}_i$ denotes the L1-normalized form of $\mathcal{H}_i$, formally defined as $\hat{\mathcal{H}}_i[c] = \mathcal{H}_i[c] / \sum_{c=1}^C \mathcal{H}_i[c] \in [0, 1]$.

Given that $\mathcal{K}(\hat{\mathcal{H}}_i || \mathcal{H}_\alpha) \in [0, +\infty)$, an upper bound is required to normalize its values into the range $[0, 1]$. Let $\mathcal{U}_1$ denotes a one-hot vector whose c th element is defined as $\mathcal{U}_1[c] = \mathbb{I}(c = \arg \min_{i \in \{1, 2, \dots, C\}} \mathcal{H}_\alpha[i])$, indicating that $\mathcal{U}_1[c] = 1$ if

and only if c corresponds to the index of the minimum value in $\mathcal{H}_\alpha$, and 0 otherwise. Then the theoretical upper bound of $\mathcal{K}(\hat{\mathcal{H}}_i || \mathcal{H}_\alpha)$ is given by

$$\mathcal{K}(\mathcal{U}_1 || \mathcal{H}_\alpha) = \begin{cases} \sum_{c=1}^C \mathcal{U}_1[c] \log \left(\frac{\mathcal{U}_1[c]}{\mathcal{H}_\alpha[c]} \right), & \text{if } \mathcal{U}_1[c] > 0 \\ 0, & \text{else.} \end{cases} \quad (16)$$

See Appendix A for a detailed proof of this upper bound.

According to the above analysis, the heterophily degree of a dataset is expressed as

$$\text{RD} = \frac{1}{C} \sum_{c=1}^C \left(\frac{1}{N_c} \sum_{i: y_i=c} \frac{\mathcal{K}(\hat{\mathcal{H}}_i || \mathcal{H}_\alpha)}{\mathcal{K}(\mathcal{U}_1 || \mathcal{H}_\alpha)} \right) \quad (17)$$

where N_c denotes the number of nodes in class c and RD lies in $[0, 1]$ (see Appendix B for detailed bound analysis).

Nonetheless, RD is currently limited to scenarios where nodes are characterized by a single feature modality and the hypergraph topology remains fixed over time. Extending RD to support multimodal node features and time-evolving hypergraph structures constitutes one of our key directions for future work.

IV. EXPERIMENTS ON UAVS

A. Experimental Setup

To demonstrate the superiority of HDHGNN, we conduct comprehensive comparative experiments against 11 state-of-the-art hypergraph methods: HGNN [6], HGNN+ [7], HyperGCN [9], JH [37], UniGCN, UniGIN, UniGAT, and UniSAGE [10], HNHN [23], ED-HNN [32], and UniG-Encoder [33]. Given that the heterophily degree varies with the choice of nearest-neighbor parameter k and hidden dimension h , we employ a grid search to thoroughly evaluate each method's performance across both datasets. Specifically, we consider $k \in \{2, 4, 6, 8, 10, 12, 14, 16, 18, 20\}$ and $h \in \{16, 32, 64, 128, 256\}$. All models are optimized

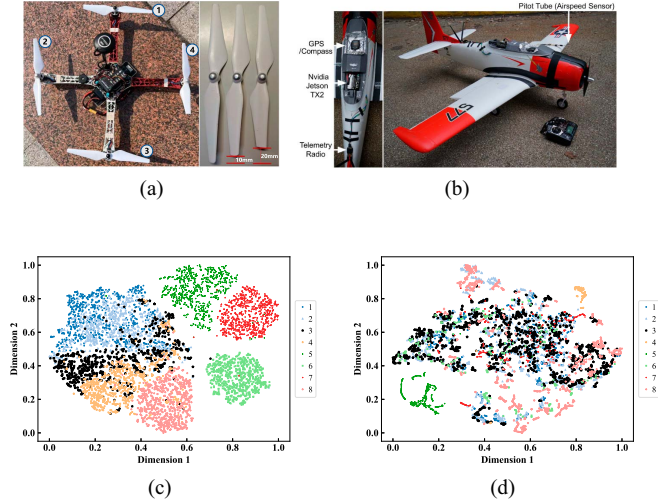


Fig. 5. UAV Platforms and t-SNE visualizations of fault distributions. (a) Quad-rotor UAV. (b) Fixed-wing UAV. (c) QUAV dataset after t-SNE dimensionality reduction. (d) ALFA dataset after t-SNE dimensionality reduction.

using the Adam optimizer, with the learning rate lr selected from $\{10^{-4}, 10^{-3}, 5 \times 10^{-3}, 10^{-2}, 2 \times 10^{-2}\}$, and the weight decay wd from $\{0, 5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}\}$. To mitigate overfitting, dropout is applied with rates $do \in \{0, 0.3, 0.5, 0.7, 0.9\}$. We employ the Kaiming Normal Initialization [38] for parameters in (8) and (11) that govern the learning of the homophily-heterophily balance through Θ_e and Θ_v . It initializes weights using a variance-scaled normal distribution (adjusted to the number of input units), ensuring consistent activation variance across layers and thus enabling stabilized gradient propagation during their adaptation. Training proceeds for up to 500 epochs, with the dataset randomly split into training and test subsets in a 7:3 ratio, and results reported as the mean and standard deviation over five runs.

The classification experiments are conducted on a hardware platform equipped with a 32-core, 64-thread CPU and 2 NVIDIA RTX 4090 GPUs. To evaluate the time efficiency of the proposed method on resource-constrained devices, we further perform runtime analysis on a laptop featuring an 8-core, 16-thread CPU (2.50 GHz), an NVIDIA RTX A4000 Laptop GPU (8 GB VRAM), and 32 GB system memory.

B. Datasets Description

This experiment uses two real-flight datasets from Quad-rotor UAV (QUAV) and fixed-wing UAV (FWUAV) for validation. As shown in Fig. 5(a), the first dataset is self-collected using a QUAV. The aircraft is equipped with onboard accelerometers and barometers that continuously record its operational status. Thirteen key variables are extracted to represent the aircraft's state. Each of the QUAV's four propeller blades is shortened by either 10 or 20 mm, yielding eight distinct fault categories with 800 samples each—resulting in a category-balanced dataset of 6400 samples.

TABLE I
FAULT DESCRIPTION OF UAV DATASETS

Category	ALFA (Fixed-Wing)	QUAV (Quad-Rotor)
1	Left aileron jammed at neutral	Propeller 1 cut by 10 mm
2	Right aileron jammed at neutral	Propeller 2 cut by 10 mm
3	Both ailerons jammed at neutral	Propeller 3 cut by 10 mm
4	Rudder jammed left	Propeller 4 cut by 10 mm
5	Rudder jammed right	Propeller 1 cut by 20 mm
6	Rudder and left aileron jammed at neutral	Propeller 2 cut by 20 mm
7	Elevator jammed at neutral	Propeller 3 cut by 20 mm
8	Engine failure	Propeller 4 cut by 20 mm

Fig. 5(c) visualizes the t-SNE projection of QUAV fault dataset. As shown, the data points for fault categories 1–4 form a more compact intermingled cluster compared with the relatively dispersed categories 5–8. This pattern reflects a higher degree of heterophily among the former group. This distinction stems from the different experimental configurations: faults 1–4 are collected under blades shortened by 10 mm, whereas faults 5–8 correspond to blades shortened by 20 mm. Considering the physical nature of QUAVs and the principles of flight dynamics, a smaller blade shorten introduces less disturbance to the flight state. Consequently, the resulting fault features tend to be more concentrated in distribution, leading to increased heterophily among the fault categories.

The second dataset is selected from the Air Lab Fault and Anomaly (ALFA) open dataset collected by Carnegie Mellon University's Robotics Institute [39]. As shown in Fig. 5(b), the FWUAV has a 2-m wingspan and is equipped with an electric engine, ailerons, flaperons, an elevator, and a rudder. The experiment employs a category-imbalanced subset comprising 5128 samples across six feature dimensions, covering eight fault categories related to the aileron, rudder, and engine. The category-wise sample counts are highly skewed, with the individual category sizes being 556, 1028, 1639, 93, 321, 272, 233, and 986, respectively. Fig. 5(d) presents the t-SNE projection of this fault dataset. The distribution reveals that the diverse real-world faults inherent to the ALFA dataset exhibit significantly stronger heterophily compared with the QUAV data shown in Fig. 5(c). Details of the fault types of both datasets are summarized in Table I.

C. Classification Performance

1) *QUAV Dataset*: The Macro F1 score, defined as the harmonic mean of precision and recall, is particularly suitable for evaluating model performance in both balanced and imbalanced class distributions. To assess the method performance across varying degrees of heterophily, Macro F1 scores are calculated as k increases from 2 to 20. For each method, to mitigate run-to-run variance, the mean and standard deviation are reported over five independent runs with different random seeds. The results are summarized in Table II. In the table, the highest value for each k is highlighted in bold, and the second-best value is underlined. Note that, as detailed in the Section IV-A, all reported F1 scores correspond to the best performance achieved

TABLE II
MACRO F1 PERFORMANCE OF METHODS ON QUAV DATASET

Method	$k = 2$	$k = 4$	$k = 6$	$k = 8$	$k = 10$	$k = 12$	$k = 14$	$k = 16$	$k = 18$	$k = 20$
	RD = 0.95	RD = 0.85	RD = 0.78	RD = 0.73	RD = 0.69	RD = 0.66	RD = 0.63	RD = 0.61	RD = 0.58	RD = 0.56
HGNN	94.00 $\pm$ 0.04	92.75 $\pm$ 0.05	92.42 $\pm$ 0.12	92.89 $\pm$ 0.10	92.92 $\pm$ 0.08	93.11 $\pm$ 0.09	92.87 $\pm$ 0.03	92.98 $\pm$ 0.04	93.14 $\pm$ 0.07	93.43 $\pm$ 0.02
HGNNP	93.55 $\pm$ 0.04	92.43 $\pm$ 0.04	92.67 $\pm$ 0.09	92.79 $\pm$ 0.03	92.96 $\pm$ 0.04	92.88 $\pm$ 0.06	92.98 $\pm$ 0.11	92.96 $\pm$ 0.09	93.13 $\pm$ 0.05	93.37 $\pm$ 0.09
HyperGCN	94.63 $\pm$ 0.06	94.41 $\pm$ 0.52	94.80 $\pm$ 0.34	94.17 $\pm$ 0.76	95.20 $\pm$ 0.62	95.48 $\pm$ 0.74	96.06 $\pm$ 0.50	96.26 $\pm$ 0.38	96.13 $\pm$ 0.65	96.16 $\pm$ 0.40
JH	84.92 $\pm$ 1.24	84.95 $\pm$ 2.38	84.80 $\pm$ 1.10	83.47 $\pm$ 3.28	82.94 $\pm$ 1.50	82.97 $\pm$ 0.99	82.70 $\pm$ 0.61	82.68 $\pm$ 2.41	80.36 $\pm$ 1.26	81.59 $\pm$ 1.36
UniGCN	93.52 $\pm$ 0.02	92.22 $\pm$ 0.12	92.50 $\pm$ 0.09	92.58 $\pm$ 0.04	92.58 $\pm$ 0.08	92.81 $\pm$ 0.07	92.79 $\pm$ 0.04	92.87 $\pm$ 0.07	93.01 $\pm$ 0.10	93.32 $\pm$ 0.06
UniGIN	95.19 $\pm$ 0.08	93.75 $\pm$ 0.17	93.24 $\pm$ 0.11	93.33 $\pm$ 0.21	93.29 $\pm$ 0.13	93.25 $\pm$ 0.20	93.00 $\pm$ 0.19	93.09 $\pm$ 0.18	92.63 $\pm$ 0.19	93.06 $\pm$ 0.22
UniGAT	93.44 $\pm$ 0.08	92.53 $\pm$ 0.11	92.83 $\pm$ 0.08	92.72 $\pm$ 0.08	93.07 $\pm$ 0.09	93.23 $\pm$ 0.17	93.08 $\pm$ 0.02	93.18 $\pm$ 0.05	93.30 $\pm$ 0.09	93.65 $\pm$ 0.13
UniSAGE	95.14 $\pm$ 0.07	93.79 $\pm$ 0.11	93.31 $\pm$ 0.05	93.03 $\pm$ 0.09	93.36 $\pm$ 0.17	93.15 $\pm$ 0.12	93.33 $\pm$ 0.19	92.98 $\pm$ 0.24	92.62 $\pm$ 0.20	92.98 $\pm$ 0.17
HNHN	93.41 $\pm$ 0.05	92.64 $\pm$ 0.12	92.84 $\pm$ 0.05	92.85 $\pm$ 0.05	93.13 $\pm$ 0.03	93.50 $\pm$ 0.11	93.31 $\pm$ 0.05	93.10 $\pm$ 0.14	93.38 $\pm$ 0.05	93.67 $\pm$ 0.04
ED-HNN	95.90 $\pm$ 0.04	95.36 $\pm$ 0.19	95.95 $\pm$ 0.34	96.80 $\pm$ 0.16	96.94 $\pm$ 0.24	96.94 $\pm$ 0.18	95.46 $\pm$ 0.14	97.02 $\pm$ 0.11	97.22 $\pm$ 0.14	97.24 $\pm$ 0.19
UniG-Encoder	96.62 $\pm$ 0.13	95.35 $\pm$ 0.19	94.75 $\pm$ 0.19	94.29 $\pm$ 0.10	94.49 $\pm$ 0.05	94.28 $\pm$ 0.11	94.28 $\pm$ 0.12	94.40 $\pm$ 0.05	94.64 $\pm$ 0.15	94.63 $\pm$ 0.18
HDHGNN	<u>96.30 $\pm$ 0.51</u>	96.84 $\pm$ 0.33	98.16 $\pm$ 0.14	96.95 $\pm$ 0.60	97.75 $\pm$ 0.77	98.41 $\pm$ 0.27	97.75 $\pm$ 0.12	97.98 $\pm$ 0.22	98.11 $\pm$ 0.23	97.95 $\pm$ 0.43

TABLE III
MACRO F1 PERFORMANCE OF METHODS ON ALFA DATASET

Method	$k = 2$	$k = 4$	$k = 6$	$k = 8$	$k = 10$	$k = 12$	$k = 14$	$k = 16$	$k = 18$	$k = 20$
	RD = 0.54	RD = 0.45	RD = 0.39	RD = 0.36	RD = 0.33	RD = 0.32	RD = 0.30	RD = 0.29	RD = 0.28	RD = 0.27
HGNN	70.28 $\pm$ 0.39	70.11 $\pm$ 0.33	70.60 $\pm$ 0.46	70.24 $\pm$ 0.42	69.08 $\pm$ 0.17	67.31 $\pm$ 0.45	65.99 $\pm$ 0.22	66.10 $\pm$ 0.27	65.55 $\pm$ 0.43	65.43 $\pm$ 0.21
HGNNP	69.63 $\pm$ 0.53	70.52 $\pm$ 0.29	70.44 $\pm$ 0.29	70.41 $\pm$ 0.62	69.43 $\pm$ 0.28	67.27 $\pm$ 0.26	66.55 $\pm$ 0.36	65.68 $\pm$ 0.34	66.130.29	65.81 $\pm$ 0.34
HyperGCN	69.82 $\pm$ 0.23	69.21 $\pm$ 0.70	68.61 $\pm$ 0.11	67.67 $\pm$ 0.82	68.05 $\pm$ 0.50	67.14 $\pm$ 0.43	67.27 $\pm$ 0.23	67.33 $\pm$ 0.26	67.63 $\pm$ 0.52	67.20 $\pm$ 0.40
JH	22.43 $\pm$ 3.73	24.80 $\pm$ 4.90	26.22 $\pm$ 0.74	25.64 $\pm$ 2.19	24.04 $\pm$ 4.48	25.72 $\pm$ 5.78	24.77 $\pm$ 6.16	22.73 $\pm$ 4.34	25.16 $\pm$ 1.64	27.23 $\pm$ 8.10
UniGCN	71.28 $\pm$ 0.63	70.61 $\pm$ 0.29	71.33 $\pm$ 0.29	70.63 $\pm$ 0.44	68.93 $\pm$ 0.31	66.93 $\pm$ 0.26	66.03 $\pm$ 0.49	65.09 $\pm$ 0.19	64.70 $\pm$ 0.18	64.71 $\pm$ 0.29
UniGIN	72.30 $\pm$ 0.49	71.98 $\pm$ 0.60	71.96 $\pm$ 0.36	71.70 $\pm$ 0.51	69.30 $\pm$ 0.40	67.45 $\pm$ 0.61	65.76 $\pm$ 0.62	63.89 $\pm$ 0.92	63.74 $\pm$ 0.53	63.75 $\pm$ 0.68
UniGAT	73.23 $\pm$ 0.62	72.14 $\pm$ 0.43	72.30 $\pm$ 0.41	72.72 $\pm$ 0.26	72.30 $\pm$ 0.20	70.94 $\pm$ 0.59	70.14 $\pm$ 0.26	69.01 $\pm$ 0.31	68.46 $\pm$ 0.50	67.86 $\pm$ 0.65
UniSAGE	72.30 $\pm$ 0.16	71.88 $\pm$ 0.67	71.90 $\pm$ 0.63	71.82 $\pm$ 0.78	70.12 $\pm$ 1.03	67.64 $\pm$ 0.85	64.75 $\pm$ 0.48	64.37 $\pm$ 0.34	63.78 $\pm$ 0.58	63.48 $\pm$ 0.24
HNHN	75.41 $\pm$ 0.59	74.54 $\pm$ 0.39	74.38 $\pm$ 0.51	73.75 $\pm$ 0.56	73.07 $\pm$ 0.37	73.14 $\pm$ 0.46	71.04 $\pm$ 0.23	70.0 $\pm$ 0.31	70.66 $\pm$ 0.33	70.12 $\pm$ 0.32
ED-HNN	80.15 $\pm$ 0.88	<u>79.15 $\pm$ 0.42</u>	<u>78.93 $\pm$ 0.64</u>	<u>77.94 $\pm$ 0.63</u>	<u>76.84 $\pm$ 0.42</u>	<u>77.13 $\pm$ 0.32</u>	<u>75.91 $\pm$ 0.32</u>	<u>75.41 $\pm$ 0.20</u>	<u>76.08 $\pm$ 0.65</u>	71.37 $\pm$ 0.61
UniG-Encoder	78.19 $\pm$ 0.38	78.01 $\pm$ 0.37	76.81 $\pm$ 0.64	75.72 $\pm$ 0.37	75.42 $\pm$ 0.65	74.83 $\pm$ 0.35	74.10 $\pm$ 0.50	72.62 $\pm$ 0.36	72.29 $\pm$ 0.26	71.67 $\pm$ 0.76
HDHGNN	<u>79.18 $\pm$ 1.44</u>	79.18 $\pm$ 1.34	79.57 $\pm$ 1.12	78.75 $\pm$ 0.88	77.96 $\pm$ 0.63	78.76 $\pm$ 0.67	76.08 $\pm$ 0.92	76.73 $\pm$ 0.63	77.03 $\pm$ 1.17	77.42 $\pm$ 1.46

across varying hidden dimensions h , learning rates lr , weight decay values wd and dropout rates do .

The results in Table II indicate that as k increases, the RD scores on the QUAV dataset decrease, reflecting an increase in the level of heterophily. As observed in this table, the proposed HDHGNN method achieves the highest mean Macro F1 score among all 11 comparative methods for every $k > 2$, with relative improvements ranging from 0.15% to 2.29%. The only exception occurs at $k = 2$, where HDHGNN yields a slightly lower score ($96.30\% \pm 0.51$) compared with best competitor UniG-Encoder ($96.62\% \pm 0.13$). However, the marginal difference of 0.32% is well within the standard deviation of the HDHGNN method, indicating statistically comparable performance of the two competitors. Moreover, when $k = 2$, each hyperedge connects exactly two nodes, effectively reducing the hypergraph to a simple graph. Consequently, the advantages of carefully designed mechanisms for hypergraphs are not fully exploitable under this setting, which partially explains the limited performance gap.

2) *ALFA Dataset*: The performance of each method on the ALFA dataset is summarized in Table III. As shown in this table, the RD values range from 0.27 to 0.54, indicating a significantly higher degree of heterophily compared with the QUAV dataset. Under these challenging conditions, HDHGNN consistently outperforms all comparative methods for $k > 2$, with mean Macro F1 improvements ranging from 0.03% to

5.75%, demonstrating its superior effectiveness and robustness across varying levels of heterophily.

D. Ablation Studies

The proposed HDHGNN method comprises two independent modules: 1) dynamic propagation from nodes to hyperedges (Dv2e); and 2) dynamic propagation from hyperedges to nodes (De2v). To evaluate the individual contribution and complementary effect of each modules, we conduct ablation studies using four variants.

- 1) *HDHGNN*: With both Dv2e and De2v modules.
- 2) *HDHGNN \ Dv2e*: Removes the Dv2e module, i.e., omits the learnable parameter $\theta_{e_i}^{(l)}$ in (6) and its matrix form $\Theta_e^{(l)}$ in (7) and (8), retaining only the De2v module.
- 3) *HDHGNN \ De2v*: Remove the De2v module, i.e., omits the learnable parameter $\theta_{v_i}^{(l)}$ in (9) and its matrix form $\Theta_v^{(l)}$ in (10) and (11), while retaining the Dv2e module.
- 4) *HDHGNN \ Dv2e \ De2v*: Removes both Dv2e and De2v modules, serving as the baseline model.

The results of the ablation study are summarized in Fig. 6. As shown, on both the ALFA and QUAV datasets, models equipped with either the Dv2e or De2v component consistently outperform the baseline (HDHGNN \ Dv2e \ De2v), indicating that each component contributes positively to the overall performance. Moreover, the full HDHGNN model achieves the best

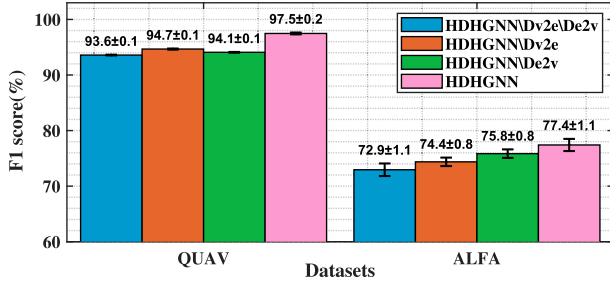


Fig. 6. Ablation study on ALFA and QUA. Variants: HDHGNN\Dv2e (w/o node-to-hyperedge dynamic propagation), HDHGNN\De2v (w/o hyperedge-to-node dynamic propagation), and HDHGNN\Dv2e\De2v (w/o both).

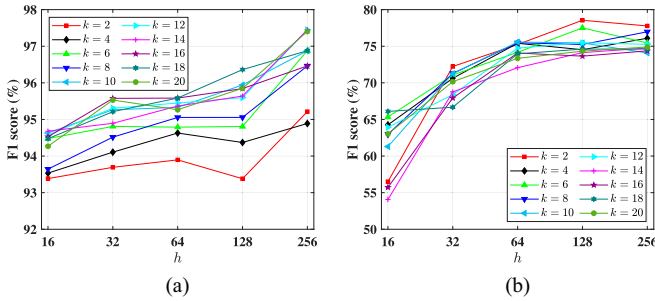


Fig. 7. Hyperparameter sensitivity analysis of HDHGNN with respect to nearest neighbor size k and hidden dimension h . (a) Macro F1 curves on the QUA dataset. (b) Macro F1 curves on the ALFA dataset.

results across all datasets, consistently surpassing all ablated variants. This demonstrates the complementary nature of the two propagation mechanisms. This performance gain from their joint integration confirms that combining Dv2e and De2v yields a synergistic effect, thereby validating the necessity of including both modules in the final method.

E. Hyperparameter Sensitivity Analysis

The HDHGNN utilizes two key hyperparameters: 1) the number of nearest neighbors k , which determines the number of nodes per hyperedge and then shapes distinct hypergraph structures; and 2) the hidden layer dimensionality h in feature aggregation, which governs the model's expressive capacity through the learnable parameters $\Theta_e^{(l)}$, $\Theta_v^{(l)}$, ultimately affecting dynamic aggregation performance. To systematically evaluate HDHGNN's sensitivity to its key hyperparameters, we conduct comparative experiments on ALFA and QUA datasets. The learning rate and weight decay are fixed at $lr = 0.001$ and $wd = 0.001$, respectively, while we vary the two core parameters: $k \in \{2, 4, 6, 8, 10, 12, 14, 16, 18, 20\}$ and $h \in \{16, 32, 64, 128, 256\}$. The Macro F1 scores under varying k and h settings on the two datasets are plotted in Fig. 7(a) and (b), respectively.

As shown in Fig. 7(a), for the QUA dataset, the F1 score curves under different k values exhibit an upward trend as h increases. Specifically, when using typical hypergraph settings where $k > 6$ and $h > 64$, all scores stabilize above 95%. This

not only demonstrates a significant performance advantage over other algorithms listed in Table II but also highlights the robustness and practicality of HDHGNN under varying hyperparameter configurations. For the ALFA dataset, as illustrated in Fig. 7(b), the mean Macro F1 scores for different k settings show a consistent upward trend with increasing h . Notably, all curves stabilize around 75% when $h \geq 64$, indicating that HDHGNN maintains stable and high performance across a wide range of hyperparameter settings. This demonstrates its strong parameter robustness and suitability for highly heterophilic scenarios.

F. Effectiveness Analysis of RD Metric

In this section, we use the highly heterophilic ALFA dataset as a representative case to compare RD against three widely adopted heterophily metrics from [36]: 1) edge homophily; 2) node homophily; and 3) class homophily. Detailed descriptions of these three metrics are provided in Appendix C. To compare metric behavior across varying nearest neighbor settings, we present radar plots depicting the performance of four evaluation metrics across the eight fault categories in the ALFA dataset, as illustrated in Fig. 8.

As shown in the plots, the proposed RD metric exhibits markedly different behavior from existing metrics in Figs. 8(b), (c), (e), and (h), while showing closer alignment in Fig. 8(d), (f), and (g), with the remaining cases falling in between. To understand the underlying causes and comparatively evaluate the strengths and limitations of each metric, we analyze two representative cases: Fig. 8(c) (corresponding to fault category 3 in ALFA) and Fig. 8(d) (category 4).

First, the data distribution in Fig. 5(d) reveals that samples from category 3 (black nodes) are heavily intermixed with multiple other classes, indicating strong distributional heterophily. Consequently, this category should receive a low score to represent high heterophily. However, as shown in Fig. 8(c), only RD consistently yields low scores across all k settings, whereas the edge, node, and class-based metrics provide significantly higher scores, failing to capture the true heterophilic nature of this category. This discrepancy arises because popular metrics treat all noncategory-3 neighbors as a single group, thereby overlooking the fine-grained coexistence of multiple distinct classes. As a result, they erroneously inflate the heterophily estimate. In contrast, RD's ability to discern such nuanced interclass structures demonstrates its fine-grained heterophily assessment capability, enabling more accurate characterization of complex data distributions.

Turning to category 4 (yellow nodes) in Fig. 5(d), we observe a compact well-separated cluster indicative of low heterophily level, and therefore expect a high heterophily score. Consistently, Fig. 8(d) shows that RD, along with other metrics, correctly assigns high scores across varying k , confirming that RD performs comparably to existing methods in simple, well-structured scenarios. Similar patterns hold across the remaining radar plots, further validating the robustness and practical utility of the proposed RD metric.

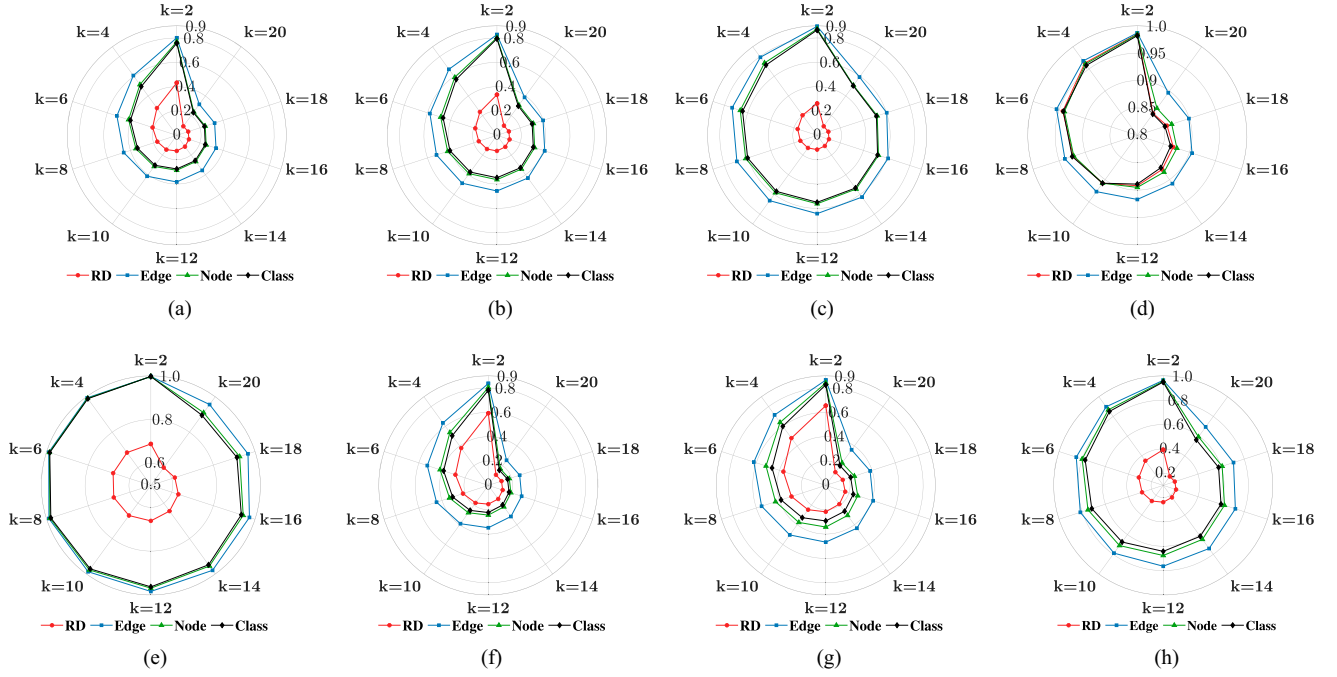


Fig. 8. Performance across heterophily metrics and k values on ALFA: a radar visualization. (a) Fault category 1. (b) Fault category 2. (c) Fault category 3. (d) Fault category 4. (e) Fault category 5. (f) Fault category 6. (g) Fault category 7. (h) Fault category 8.

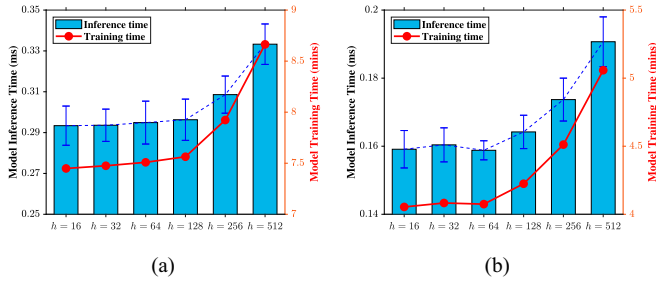


Fig. 9. HDHGNN Training and inference time consumption with various h settings. (a) On QUAU dataset. (b) On ALFA dataset.

G. Time Efficiency Analysis

To evaluate the time efficiency of HDHGNN on resource-constrained devices, we conduct a comprehensive runtime analysis on a laptop with hardware specifications as described in Section IV-A. All experiments are performed under the largest nearest neighbor size ($k = 20$), which induces the highest computational complexity and thus represents the worst-case scenario in terms of training and inference latency. Furthermore, to assess the impact of hidden dimension h , we measure the total training time for 500 epochs and the average inference time (over 500 runs) along with its standard deviation on both datasets, with $h \in \{16, 32, 64, 128, 256, 512\}$. The results are visualized in Fig. 9.

As shown in Fig. 9(a), on the QUAU dataset, both online inference and offline training times increase monotonically with h . Specifically, the average single inference latency ranges

from 0.2934 ± 0.0096 ms to 0.3333 ± 0.0099 ms. A full offline training run (500 epochs) on the constrained laptop takes between 7.45 and 8.66 min. During training, system memory usage fluctuates between 3.4 GB and 4.0 GB, while GPU memory consumption remains modest at 3.8–4.4 GB. On the ALFA dataset [Fig. 9(b)], inference is even faster, with per-inference latency ranging from 0.1591 ± 0.0055 ms to 0.1907 ± 0.0073 ms. Full training requires only 4.05–5.06 min, with memory usage of 2.6–3.4 GB and GPU memory of 2.7–3.3 GB. These results demonstrate that HDHGNN achieves millisecond-level inference speed and sub-9-min training time even on modest hardware, highlighting its strong practicality and real-time applicability. Notably, in real-world deployment, offline training is typically performed once and can be trained on more powerful machines, further reducing the effective training overhead.

V. CONCLUSION

The heterophily problem represents one of the most challenging issues in graph-based learning. Conventional hypergraph frameworks are primarily built upon the homophily assumption and lack the capability to address heterophily in real-world scenarios. To bridge this gap, this article introduces a heterophily-based dynamic structure and proposes the HDHGNN method, accompanied by a novel relative entropy-based metric, RD, for evaluating heterophily degree. In HDHGNN, the propagation process is decoupled into two causally-related subprocedures, with dedicated attention mechanisms designed to facilitate dynamic aggregation under the heterophily assumption. By extending the conventional hypergraph framework,

HDHGNN exhibits strong adaptability to both heterophilic and homophilic data distributions. Its effectiveness is validated on two real-world UAV flight datasets, where it achieves significant performance improvements compared with 11 state-of-the-art methods.

REFERENCES

- [1] C. Yang, K. Zhou, and J. Liu, "Supergraph: Spatial-temporal graph-based feature extraction for rotating machinery diagnosis," *IEEE Trans. Ind. Electron.*, vol. 69, no. 4, pp. 4167–4176, Apr. 2022.
- [2] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Y. Guo and F. Farooq, Eds., 2018, pp. 974–983.
- [3] X. Wang, Y. Ye, and A. Gupta, "Zero-shot recognition via semantic embeddings and knowledge graphs," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 6857–6866.
- [4] G. Yang, H. Tao, T. Yu, R. Du, and Y. Zhong, "Online fault diagnosis of harmonic drives using semisupervised contrastive graph generative network via multimodal data," *IEEE Trans. Ind. Electron.*, vol. 71, no. 3, pp. 3055–3063, Mar. 2024.
- [5] X. Zheng et al., "Graph neural networks for graphs with heterophily: A survey," 2024, *arXiv:2202.07082*.
- [6] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," in *Proc. 33rd AAAI Conf. Artif. Intell.*, 2019, pp. 3558–3565.
- [7] Y. Gao, Y. Feng, S. Ji, and R. Ji, "Hgnn⁺: General hypergraph neural networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3181–3199, Mar. 2023.
- [8] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," 2019, *arXiv:1903.02428*.
- [9] N. Yadati, M. Nimishakavi, P. Yadav, V. Nitin, A. Louis, and P. Talukdar, "HyperGCN: A new method of training graph convolutional networks on hypergraphs," 2019, *arXiv:1809.02589*.
- [10] J. Huang and J. Yang, "UniGNN: A unified framework for graph and hypergraph neural networks," in *Proc. 30th Int. Joint Conf. Artif. Intell.*, Z. Zhou, ed., 2021, pp. 2563–2569.
- [11] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 7793–7804, 2020.
- [12] J. Wang, Y. Guo, L. Yang, and Y. Wang, "Heterophily-aware graph attention network," *Pattern Recognit.*, vol. 156, 2024, Art. no. 110738. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320324004898>
- [13] N. Veldt, A. R. Benson, and J. M. Kleinberg, "Higher-order homophily is combinatorially impossible," 2021, *arXiv:2103.11818*.
- [14] S. Luan et al., "The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges," 2024, *arXiv:2407.09618*.
- [15] Y. Zheng, H. Zhang, V. Lee, Y. Zheng, X. Wang, and S. Pan, "Finding the missing-half: Graph complementary learning for homophily-prone and heterophily-prone graphs," in *Proc. 40th Int. Conf. Mach. Learn.*, 2023.
- [16] Y. Song, C. Zhou, X. Wang, and Z. Lin, "Ordered GNN: Ordering message passing to deal with heterophily and over-smoothing," in *Proc. 11th Int. Conf. Learn. Representations*, 2023.
- [17] R. Lei, Z. Wang, Y. Li, B. Ding, and Z. Wei, "EvenNet: Ignoring odd-hop neighbors improves robustness of graph neural networks," in *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 4694–4706, 2022.
- [18] L. Wu et al., "Beyond homophily and homogeneity assumption: Relation-based frequency adaptive graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 8497–8509, Jun. 2024.
- [19] J. Guo, K. Huang, R. Zhang, and X. Yi, "ES-GNN: Generalizing graph neural networks beyond homophily with edge splitting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 11345–11360, Dec. 2024, doi: 10.1109/TPAMI.2024.3459932.
- [20] J. Chen, S. Chen, J. Gao, Z. Huang, J. Zhang, and J. Pu, "Exploiting neighbor effect: Conv-agnostic GNN framework for graphs with heterophily," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 35, no. 10, pp. 13383–13396, Oct. 2024, doi: 10.1109/TNNLS.2023.3267902.
- [21] J. Park, S. Yoo, J. Park, and H. J. Kim, "Deformable graph convolutional networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 7, 2022, pp. 7949–7956.
- [22] J. Jiang, Y. Wei, Y. Feng, J. Cao, and Y. Gao, "Dynamic hypergraph neural networks," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2019.
- [23] Y. Dong, W. Sawin, and Y. Bengio, "HNHN: Hypergraph networks with hyperedge neurons," 2020, *arXiv:2006.12278*.
- [24] Y. Wu, S. Lan, Z. Cai, M. Fu, J. Li, and S. Wang, "SCHG: Spectral clustering-guided hypergraph neural networks for multi-view semi-supervised learning," *Expert Syst. Appl.*, vol. 277, Jun. 2025, Art. no. 127242.
- [25] J. Liao, J. Yan, and Q. Tao, "DualhGNN: A dual hypergraph neural network for semi-supervised node classification based on multi-view learning and density awareness," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2023, pp. 1–10.
- [26] J. Liao, J. Yan, Q. Tao, E. Zhang, and Y. Zhang, "A novel hypergraph neural network combining multi-view learning with density awareness," *Pattern Recognit.*, vol. 168, Dec. 2025, Art. no. 111775.
- [27] Z. Liu, B. Tang, Z. Ye, X. Dong, S. Chen, and Y. Wang, "Hypergraph transformer for semi-supervised classification," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2024, pp. 7515–7519.
- [28] H. Zhang, S. Wang, Z. Hu, Y. Qi, Z. Huang, and J. Guo, "I2hgnn: Iterative interpretable hypergraph neural network for semi-supervised classification," *Neural Netw.*, vol. 183, Mar. 2025, Art. no. 106929.
- [29] M. Yin, J. Gao, and Z. Lin, "Laplacian regularized low-rank representation and its applications," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 3, pp. 504–517, Mar. 2016.
- [30] S. Bai, F. Zhang, and P. H. Torr, "Hypergraph convolution and hypergraph attention," *Pattern Recognit.*, vol. 110, 2021, Art. no. 107637.
- [31] D. Arya, D. K. Gupta, S. Rudinac, and M. Worring, "Adaptive neural message passing for inductive learning on hypergraphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 1, pp. 19–31, Jan. 2025.
- [32] P. Wang, S. Yang, Y. Liu, Z. Wang, and P. Li, "Equivariant hypergraph diffusion neural operators," in *Proc. 11th Int. Conf. Learn. Representations*, 2023, [Online]. Available: <https://openreview.net/forum?id=RTjKoscNnd>
- [33] M. Zou et al., "UNIG-encoder: A universal feature encoder for graph and hypergraph node classification," *Pattern Recognit.*, vol. 147, Mar. 2024.
- [34] B. Nguyen, L. Sani, X. Qiu, P. Lio, and N. D. Lane, "Sheaf hypernetworks for personalized federated learning," 2024, *arXiv:2405.20882*.
- [35] M. K. Hayat, S. Xue, J. Wu, and J. Yang, "Heterogeneous hypergraph embedding for node classification in dynamic networks," *IEEE Trans. Artif. Intell.*, vol. 5, no. 11, pp. 5465–5477, Nov. 2024, doi: 10.1109/TAI.2024.3450658.
- [36] O. Platonov, D. Kuznedelev, A. Babenko, and L. Prokhorenkova, "Characterizing graph datasets for node classification: Homophily-heterophily dichotomy and beyond," *Adv. Neural Inf. Process. Syst.*, vol. 36, p. 2023, 2023.
- [37] S. Ji, Y. Feng, R. Ji, X. Zhao, W. Tang, and Y. Gao, "Dual channel hypergraph collaborative filtering," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, New York, NY, USA: ACM, 2020, pp. 2020–2029.
- [38] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. on Comput. VISION (ICCV)*, 2015, pp. 1026–1034, doi: 10.1109/ICCV2015.123.
- [39] A. Keipour, M. Mousaei, and S. A. Scherer, "ALFA: A dataset for UAV fault and anomaly detection," *Int. J. Robot. Res.*, vol. 40, nos. 2–3, pp. 515–520, 2021.



Shaojun Liang received the M.S. degree in control science and engineering from Wuhan University, Hubei, China, in 2019. He is currently working toward the Ph.D. degree in control science and engineering with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, China.

His research interests include hypergraph, graph knowledge, fault diagnosis, and Graph-based RAG and agent technology in large models.



Ying Zheng (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in control theory and engineering from Huazhong University of Science and Technology, Wuhan, China, in 1997, 2000, and 2003, respectively.

She has been a Visiting Scholar with the University of Southern California, Los Angeles, CA, USA, and a Faculty Member with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Huazhong, China. Her research interests include industrial process monitoring, fault diagnosis, and process control.



Housheng Su received the B.S. degree in automatic control and the M.S. degree in control theory and control engineering from Wuhan University of Technology, Wuhan, China, in 2002 and 2005, respectively, and the Ph.D. degree in control theory and control engineering from Shanghai Jiao Tong University, Shanghai, China, in 2008.

From 2008 to 2010, he was a Postdoctoral Researcher with the Department of Electronic Engineering, City University of Hong Kong, Kowloon Tong, Hong Kong. Since 2014, he has been a

Full Professor with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, China. His research interests include multiagent coordination control theory and its applications to autonomous robotics and mobile sensor networks.